CASE STUDY / 2026

Collabs+ Community SaaS

A modular, security-focused backend for founders, investors, service providers, events, bookings, community, and auditable credits.

FastAPI | Python 3.11 | PostgreSQL | SQLAlchemy | Redis | Celery | Docker

PREPARED BY

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

SECTION 00

Contents

A technical walkthrough of the problem, architecture, implementation, evidence, and next stage.

01	**Project Overview**	Page 3
02	**Platform Architecture**	Page 4
03	**Domain & Integrity Model**	Page 5
04	**The Engineering Challenge**	Page 6
05	**Implementation Approach**	Page 7
06	**Quality & Verification**	Page 8
07	**Outcome & Next Stage**	Page 9

Evidence standard

This case study reflects the implemented backend through Phase 6D and its documented 57-test regression suite. The frontend, production deployment, CI/CD, and payment processing are explicitly outside the delivered boundary.

SECTION 01

Project Overview

A modular-monolith backend that coordinates multi-role onboarding, collaboration workflows, community interactions, and internal credits with transactional integrity.

Collabs+ is a community-focused SaaS backend for investors, founders, service providers, and administrators. It provides verified onboarding, role-specific profiles, bookings, events, attendance, referrals, credits, notifications, community posts, direct messages, blog publishing, background tasks, caching, and administrative operations.

The system begins as one FastAPI deployment but separates business modules cleanly so domains can move to services later only when justified. PostgreSQL owns relational integrity and concurrency-sensitive rules; Redis supports caching and Celery; JWT access and refresh tokens, bcrypt, approval gates, and RBAC protect every normal feature route.

57	18	4	6D
pytest regression tests passed	implemented backend modules	Docker services	current completed phase

Modular, not fragmented

A clear business-domain structure keeps Phase 1 operable without premature microservices.

Transactional credits

Balances change only through backend operations, atomic transactions, idempotency, and immutable ledger entries.

Approval-gated access

Registration, verification, role selection, profile completion, and admin approval form one explicit onboarding lifecycle.

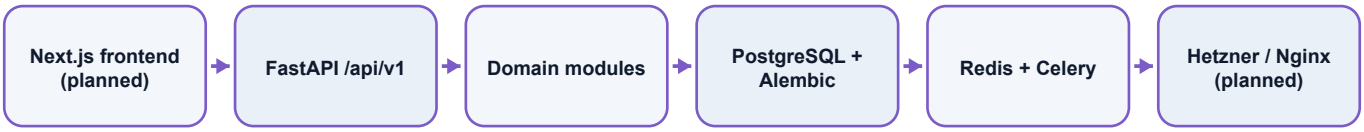
Measured scope

Unimplemented frontend, payments, real-time messaging, production infrastructure, and CI/CD remain visible as follow-up work.

SECTION 02

Platform Architecture

Docker Compose runs the async API, PostgreSQL, Redis, and a Celery worker while business modules share one controlled transaction boundary.



Layer	Implemented responsibility
API	FastAPI async routers expose versioned REST endpoints, Pydantic v2 contracts, dependency-based authorization, and structured error behavior.
Domain layer	Auth, users, role profiles, bookings, events, referrals, credits, notifications, community, messaging, blog, cache, tasks, and admin remain separate modules.
Persistence	SQLAlchemy 2.x async, asyncpg, PostgreSQL 15, Alembic, constraints, transactions, and exclusion rules protect relational and concurrency behavior.
Background work	Redis 7 supports cache isolation and Celery 5.6 queues asynchronous jobs without adding a second broker.
Runtime	Docker Compose binds backend, worker, PostgreSQL, and Redis locally; production Nginx, Hetzner, and Neon wiring remains planned.

Scale path Module boundaries are designed for later extraction, but the current system intentionally keeps one deployable backend so transactions, testing, and operations stay simple during product discovery.

SECTION 03

Domain & Integrity Model

Each business workflow has explicit roles, lifecycle states, database constraints, and audit records rather than generic CRUD behavior.

Identity and profiles

Email verification, access/refresh tokens, admin approval, and generic plus investor/founder/service-provider profiles define platform access.

Bookings and events

State transitions, attendance, check-in, discovery, and PostgreSQL overlap prevention protect scheduling and event operations.

Referrals and credits

Qualification, fraud flags, reward rules, balances, ledger entries, spending primitives, and admin adjustments remain auditable.

Community and content

Posts, likes, comments, direct conversations, notifications, public blog content, and moderation support engagement workflows.

Domain	Invariant	Implementation
Platform access	Active + verified + approved	Shared authorization dependency on normal feature routes
Booking	No overlapping provider slots	PostgreSQL GiST exclusion constraint
Credits	No direct balance mutation	Atomic backend transaction + immutable ledger + idempotency
Roles	Admin cannot be self-assigned	Role-gated endpoints and protected admin operations
Messaging	Private conversations remain private	Membership checks; admins do not receive DM read access

The backend exposes all endpoints under /api/v1. Shared authorization dependencies separate a valid token from full platform access, while module-level rules add role and ownership checks without duplicating authentication logic.

SECTION 04

The Engineering Challenge

Community features look independent, but user state, roles, scheduling, credits, notifications, and auditability interact across every module.

The engineering problem was to keep these workflows consistent without creating a tightly coupled monolith or a prematurely distributed system. Concurrency-sensitive operations also had to survive simultaneous requests, retries, and partial background work.

Lifecycle complexity

A user can be registered but unverified, verified but unapproved, inactive, rejected, or active with one of several business roles.

Schedule concurrency

Two requests can target the same provider slot, so application checks alone cannot prevent double booking.

Credit integrity

Rewards and spending require balance consistency, source attribution, idempotency, and corrections through compensating records.

Module interaction

Events trigger attendance, rewards, notifications, and admin views while preserving one transactionally coherent outcome.

Async reliability

Background tasks and cache invalidation must not silently diverge from PostgreSQL, the authoritative data store.

Product uncertainty

Payments, real-time messaging, production infrastructure, and some credit rules were deliberately deferred until requirements mature.

Integrity rule Business state changes are not trusted to UI order. Database constraints, row locks, idempotency keys, and transaction-scoped operations protect invariants when requests arrive concurrently.

SECTION 05

Implementation Approach

The backend was delivered phase by phase, with each domain added behind shared security conventions and a growing regression suite.

1. Secure foundation

Add configuration, password hashing, JWT access/refresh tokens, verification, RBAC, and admin approval.

2. Role profiles

Model shared user data plus investor, founder, and service-provider profiles with role-specific authorization.

3. Collaboration workflows

Implement bookings, event lifecycle, registration, attendance, check-in, search, and rescheduling.

4. Economic primitives

Build referral qualification, fraud flags, credit rewards, ledger, balance, spending primitive, and admin adjustment.

5. Engagement domains

Add notifications, community posts/likes/comments, direct messaging, and blog publishing/moderation.

6. Operability

Wire Redis caching, Celery jobs, Docker services, Alembic migrations, admin dashboards, and regression tests.

Concurrency pattern

Booking overlap is blocked by a PostgreSQL exclusion constraint across provider and time range. Credits use locked balance updates, immutable ledger records, and idempotency so duplicate requests cannot issue the same reward twice.

- The API and worker import the same centralized ORM model registry so relationship resolution remains consistent across entry points.
- Cache keys use a Redis database isolated from the Celery broker, reducing accidental coupling between background work and response caching.
- Permanent pytest coverage includes database constraints, API behavior, security, concurrency, transactions, caching, and background-job boundaries.

SECTION 06

Quality & Verification

Regression evidence combines API tests with database-level constraints, migration checks, and container health rather than relying on endpoint smoke tests alone.

Evidence	Verified result	Why it matters
Regression suite	57 pytest tests passed	Auth, domains, constraints, transactions, cache behavior, and administrative controls stay protected.
Schema state	Alembic head edc7f331d555; check clean	ORM models and migration history are synchronized.
Runtime	4 healthy Docker services	API, worker, PostgreSQL, and Redis run together in the documented local topology.
Phase delivery	5A-5Z and 6A-6D complete	Feature history and current boundary are documented in one project state record.
Security	JWT, bcrypt, RBAC, approval gates	Normal feature access requires active, verified, approved users and domain ownership.

Current boundary

The frontend directory is still a placeholder. Production deployment, Nginx/Hetzner/Neon wiring, CI/CD, performance testing, Celery beat scheduling, Stripe/payments, and Phase 7 are not complete and are not presented as delivered.

- Credits are internal integer points only; they are not money, withdrawable, transferable, or a payment method.
- Messaging is REST-only and attachment-free today, although the data model can support later real-time delivery.
- Search uses PostgreSQL ILIKE and remains appropriate for current scale; full-text search is a later optimization.

SECTION 07

Outcome & Next Stage

A broad product backend now shares one coherent security model, transaction strategy, migration history, and regression suite.

Faster product growth	Safer operations	Clear scale path
New domains reuse proven auth, transaction, and testing conventions.	Concurrency and credit integrity are enforced below endpoint logic.	Modules can be extracted only when load and ownership justify it.

Collabs+ demonstrates how a feature-rich community product can remain technically disciplined. The system delivers substantial backend scope without hiding incomplete product surfaces, and it leaves production infrastructure and frontend work as explicit next-stage decisions.

NEXT STAGE

- Build the Next.js frontend against the versioned API contracts and complete user-facing product journeys.
- Finalize credit and referral business rules, then add the first approved spending actions.
- Introduce CI/CD, production infrastructure, observability, performance testing, and deployment runbooks before launch.

LET'S TALK

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

Stack: FastAPI | Python 3.11 | PostgreSQL | SQLAlchemy | Redis | Celery | Docker